

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 3 – Comparative Analysis

Course Code:	CSA0309	Course Title:	Data Structures
CO Assessed:	CO4 – Articulation (BL3)	Assessment:	Assessment Tool 3 – Comparative Analysis
Weightage:	30% of CIA	Total Marks:	100
CO4 Statement:	Apply hashing strategies for efficient data storage and sorting algorithms for arrangement of data.		
Student Name:	N. Adi Narayana	Reg. No.:	198525346
Section / Batch:	ATML	Date:	28/08/2026

Code of Conduct

I N. Adi Narayana (198525346) (Name / Reg No)
certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.
Signature of the Student: N. Adi

Instructions

- This test contains 5 Comparative Analysis questions. Total: 100 Marks.
- When a student answer using comparative analysis should be based on four requirements: time complexity, space complexity advantages & limitations, real-world scenarios and operations.
- No negative marking. Unanswered questions carry zero marks.
- No DTP printouts or electronic devices permitted. They should provide written materials.

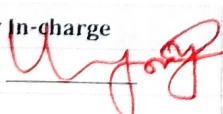
Section / Topic	Focus	Q Nos	Marks
Hashing	Linear Probing & Separate Chaining	1	20
Sorting	Merge Sort & Quick Sort	2	20
Hashing	Quadratic Probing and Double Hashing	3	20
Sorting Algorithms	Complexity Analysis	4	20
Quick Sort	Recursive and Iterative implementations	5	20
GRAND TOTAL:			100 Marks

Annexure A – Comparative Analysis

1. Compare Linear Probing and Separate Chaining for collision handling in hashing based on:
(20 Marks)
 - a) Clustering effect
 - b) Memory usage
 - c) Performance under high load factor
 - d) Which method is more suitable for large-scale applications and why?
2. Compare quick sort and merge sort for sorting a large linked list of 10 million nodes. Analyze memory requirements (in-place vs. extra space), cache behavior, and time complexity. Which algorithm is preferable for linked lists and why? How does the answer change for arrays?
(20 Marks)
3. Compare Quadratic Probing and Double Hashing in terms of:
(20 Marks)
 - a) Clustering issues
 - b) Collision resolution efficiency
 - c) Suitability for dense hash tables
4. Compare Best-case and Worst-case time complexity of sorting algorithms based on the following:
(20 Marks)
 - a) Practical significance
 - b) Impact on algorithm selection
5. Compare Recursive and Iterative implementations of Quick Sort in terms of: (20 Marks)
 - a) Space complexity (stack usage)
 - b) Ease of implementation
 - c) Risk of stack overflow

Rubrics for Calculation

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Concept Understanding	20	Demonstrates complete and accurate understanding of all data structures with advanced insights	Shows clear understanding with minor gaps	Basic understanding; some misconceptions present	Limited or incorrect understanding
Comparative Analysis Depth	20	Thorough, multi-dimensional comparison with strong justification and critical evaluation	Good comparison with relevant factors considered	Limited comparison; lacks depth or justification	Minimal or incorrect comparison
Use of Parameters (Time/Space/Operations)	30	All parameters analysed accurately with complexity discussion	Most parameters covered correctly	Few parameters considered; some inaccuracies	Parameters missing or incorrect
Critical Thinking	20	Strong justification of why one structure is better in given contexts	Some justification present	Limited reasoning	No critical reasoning
Clarity	10	Exceptionally well-structured, logical flow, clear tables/diagrams	Well-organized with minor issues	Some organization issues; lacks clarity	Poor structure and readability

Faculty In-charge	Course Coordinator	Program Director
Signature: 	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

Collision Handling

Linear Probing

* clustering effect suffers heavily from primary clustering. Long blocks of occupied slots build up cascading search times

⇒ Memory Usage Highly memory efficient. Data is stored entirely inside the table array with zero pointer

⇒ High load factor performance decreases the Insertion find next empty slot

⇒ Average $O(1)$, worst $O(n)$ for search. The Deletion is more difficult

Separate chaining

* No clustering effect. Collisions form isolated local chains without affecting unrelated hash locations

* Memory Usage Higher memory overhead. Requires dynamic memory allocation for pointers and nodes in every linked list

⇒ Handles high load better
Adds element to chain

⇒ The Search Average $O(1)$, worst $O(n)$. If Deletion is easier.

②

Quick Sort vs Merge Sort for a large linked list

Quick Sort

Time Complexity:

Guaranteed $O(n \log n)$ in worst, average, and best cases

Merge Sort

Average case is $O(n \log n)$ but worst-case degrades to $O(n^2)$ if pivots are poorly chosen

memory Requirements
⇒ $O(\log n)$ auxiliary space for the recursive call stack. It operation in-place for linked lists

Cache Behavior
⇒ Both suffer from poor cache performance on the linked list.

⇒ Usually not stable

⇒ Extra space usually less

⇒ $O(\log n)$ average call stack space. but can degrade to $O(n)$ in the worst case

⇒ Both suffer from poor cache performance on linked lists.

⇒ stable

⇒ Extra space Required extra space.

③

Quadratic Probing

⇒ The clustering Reduces Primary clustering but may have Secondary clustering

⇒ Collision resolution checks Position using quadratic Jumps

⇒ Efficiency (good)

⇒ Dense hash tables less suitable when table very

⇒ Average time $O(1)$

Double Hashing

⇒ The minimizes clustering

⇒ uses a second hash function

⇒ Generally better

⇒ More suitable

⇒ Average time $O(1)$

Quadratic Probing:

Simple and reduces primary clustering

Double Hashing:

Gives better distribution and reduces clustering.

Best-Case Complexity

Worst-case Complexity

Primary focus:

⇒ Efficient on optimal inputs

⇒ Big Omega (Ω)

⇒ Minimum time required

⇒ Practical significance shows
Performance on favorable
input

⇒ Identifies Early Exit
Opportunities are some
algorithms can detect
an already sorted
array

⇒ $O(n)$ Data already
sorted

⇒ Safety and performance the
guarantees

⇒ Big $O(b)$

⇒ maximum time required

⇒ Shows performance on
unfavorable input

⇒ Guarantees upper bound
it provides an absolute
performance guarantee
Security Defences

Ex: $O(n^2)$ (Pivot splits
data highly unevenly)

Recursive Quick sort

stack
uses the system's automatic memory (call stack)

⇒ Code simplicity:
Very easy to write and understand

⇒ High risk of crashing (stack overflow) on large or sorted data

⇒ Good performance $O(\log n)$

⇒ Space Complexity Average

Iterative Quick Sort

⇒ Uses a manual stack created by the programmer

⇒ Harder to write; requires managing indices manually

⇒ Very safe, uses larger & gen memory (Heap) so, it rarely crashes

⇒ Good performance

→ Average $O(\log n)$, worst $+ O(n)$